

PRÄSENTATIONSLEITFADEN & REDESKRIPTE

Wort-für-Wort-Vortragstext & Fragenkatalog für das IHK-Fachgespräch

Projekt: Smart Restaurant – Digitales Bestell- und Verwaltungssystem

Prüfungsprojekt:	Smart Restaurant (Another Great Solution GmbH / Team „Der Dreier“)
Präsentationsdauer:	Exakt 20 Minuten (strikte Zeiteinhaltung: ca. 80–85 Sek. pro Folie)
Fachgespräch:	10 bis 15 Minuten vertiefende Fachdiskussion mit dem IHK-Prüfungsausschuss
Inhalt des Dokuments:	Vollständige Redeskripte mit Regieanweisungen, Timing, Folienleitfaden & Prüfer-Q&A
Dokumentenstand:	Oktober 2026 Version 2.1 (Erweitert um Redeskripte)

Dieses Dokument begleitet die PowerPoint-Präsentation `abgabe/Smart_Restaurant_IHK_Praesentation.pptx`.
Es liefert den drei Prüflingen einen professionellen Leitfaden zur optimalen Vorbereitung auf den mündlichen Prüfungsteil.

1. Moderations- & Verhaltensregeln für den 20-Minuten-Vortrag

1. Das Staffellauf-Prinzip (Nahtlose Übergaben):

Die Präsentation ist in drei Blöcke à ca. 6,5 Minuten aufgeteilt. Die Übergabe erfolgt fließend per Staffelstab-Moderation: Der abgebende Sprecher kündigt das Folgethema mit einem Überleitungssatz an, der übernehmende Sprecher knüpft mit einem Dankessatz an und führt ohne Unterbrechung fort.

2. Zeichenerklärung im Redeskript:

- **[Regieanweisung in eckigen Klammern]**: Physische Aktionen, Gestik, Blickkontakt oder Verweise auf Folieninhalte.
- **//** : Kurze Sprechpause (ca. 1 Sekunde) zur gedanklichen Strukturierung und Akzentuierung von Kernbotschaften.
- **Fettgedruckte Fachbegriffe**: Bewusst betonen und mit Selbstverständlichkeit aussprechen.

2. Foliengenaue Redeskripte & Erwartbare Prüferfragen

Folie 1 Titelfolie – Projektvorstellung & Eckdaten

00:00 – 01:00 min

Wort-für-Wort-Redeskript:

[Offene Körperhaltung, freundlicher und verbindlicher Blickkontakt zu allen Prüfern]

„Sehr geehrte Mitglieder des Prüfungsausschusses, herzlich willkommen zu unserer Projektpräsentation. // Wir präsentieren Ihnen heute unser Projekt ‚**Smart Restaurant**‘ – die Konzeption und Realisierung eines digitalen Bestell- und Verwaltungssystems für einen regionalen Gastronomiebetrieb. // Im Auftrag unseres Szenario-Kunden hat unser Projektteam ‚Der Dreier‘ bei der Another Great Solution GmbH das Vorhaben in einem straffen Zeitrahmen von **10 Arbeitstagen** und exakt **320 Personenstunden** umgesetzt. *[Geste auf die Kennzahlen-Chips]* Das Ergebnis ist eine vollständige, lauffähige Software in **Java 17** mit relationaler SQL-Datenbankanbindung, einer automatisierten Testsuite mit 100 % Erfolgsquote sowie einem begleitenden ISMS-Sicherheitsbericht nach **ISO/IEC 27001**. // In den kommenden 20 Minuten nehmen wir Sie mit durch unsere wirtschaftlichen Entscheidungen, die technische Architektur bis hin zum Soll-Ist-Controlling.“

? **Prüferfrage:** „Wie kam die Teamgröße von 4 Personen im Szenario bei 3 Vortragenden zustande?“

💡 **Antwort:** „Das Szenario Another Great Solution GmbH kalkuliert mit 4 Rollen-Äquivalenten (Projektleitung, Analyse, Backend, Frontend) über 320 Personenstunden, um alle Gewerke abzubilden. In unserem Realteam haben wir die Verantwortungsbereiche auf uns drei Prüflinge aufgeteilt und arbeitsteilig umgesetzt.“

Folie 2 Agenda & Gliederung

01:00 – 01:45 min

Wort-für-Wort-Redeskript:

[Deutet mit offener Hand auf die drei nummerierten Karten auf der Folie]

„Um Ihnen einen transparenten Überblick zu geben, haben wir unsere 20-minütige Präsentation in drei inhaltlich ausgewogene Blöcke unterteilt: // Wir starten in **Block 1** mit der Ausgangslage, der methodischen Make-or-Buy-Entscheidung, dem Phasenplan und der detaillierten Zuschlagskalkulation inklusive Energiekosten. // Im **zweiten Block** widmen wir uns ab Minute 6 der technischen Substanz: Wir zeigen Ihnen das prozessuale Routing und Zustandsmanagement im Quellcode, das relationale Datenmodell in der 3. Normalform sowie unsere Java 17 Transaktions- und DAO-Architektur. // Im **dritten Block** ab Minute 13 fokussieren wir uns auf die Zuverlässigkeit: Unsere automatisierte Testsuite, das Informationssicherheitskonzept nach ISO 27001, den Soll-Ist-Vergleich und die Roadmap für künftige Erweiterungen. // Lassen Sie uns direkt mit der Ausgangssituation unseres Kunden beginnen.“

? **Prüferfrage:** „Warum haben Sie dem Qualitäts- und Sicherheitsbereich einen so großen zeitlichen Anteil eingeräumt?“

💡 **Antwort:** „In der Gastronomie führen Systemausfälle im laufenden Abendbetrieb zu direktem Umsatzverlust und Reputationsschäden. Daher war ein belastbares Test- und Sicherheitskonzept ein zwingendes Kernkriterium der Fachabnahme.“

**Wort-für-Wort-Redeskript:**

[Blick zur linken roten Box der Folie]

„Betrachten wir die Ausgangslage unseres Gastronomiebetriebs: Bislang basierte die gesamte Bestellaufnahme auf handschriftlichen Notizzetteln und Durchschreibeblocks. // Das führte in Stoßzeiten zu gravierenden Problemen:

[Zählt an den Fingern auf] Erstens Lesefehler in der Küche durch unleserliche Handschriften, was zu Fehlzubereitungen führte. Zweitens enorme Laufwege: Jede Servicekraft musste physisch vom Gästetisch zur Küche und zur Bar laufen, um Bons abzugeben. Und drittens ein reales Abrechnungsrisiko: Ging ein Papierbon im Trubel verloren, bedeutete das einen direkten Umsatzausfall bei der Bezahlung. // *[Blick zur rechten grünen Box]* Unser **SOLL-Konzept** löst diese Schwachstellen vollständig ab: Bestellungen werden digital an Terminals erfasst und in Echtzeit automatisiert getrennt an die Küche und die Bar übertragen. Alle Vorgänge sind transparent protokolliert, was die Durchlaufzeiten halbiert und eine fehlerfreie, revisionssichere Abrechnung garantiert.“

**Prüferfrage: „Haben Sie den Zeitverlust der Servicekräfte im Vorfeld quantifiziert?“**

Antwort: „Ja, in der Anforderungsanalyse ergab eine Stichprobe im IST-Zustand ca. 2 bis 3 Minuten Laufweg pro Bestellung allein für die Bon-Überbringung. Bei 150 Tischen pro Abend summiert sich das auf über 5 Stunden reine Laufzeit, die nun für die Gästebetreuung frei werden.“

 Wort-für-Wort-Redeskript:

[Zeigt auf die Kriterien-Tabelle auf Folie 4]

„Bevor wir mit der Entwicklung begonnen haben, stand eine fundamentale betriebswirtschaftliche Entscheidung an: **Make or Buy?** // Warum kaufen wir keine fertige Standard-Gastro-Cloud-Software? // Unsere Nutzwert- und TCO-Analyse hat ein klares Bild ergeben: Standard-SaaS-Lösungen binden den Gastronomen in ein dauerhaftes Vendor-Lock-in. Bei 3 Terminals fallen pro Monat ca. 177 € an, was sich auf über **2.124 € Lizenzgebühren pro Jahr** summiert – exklusive Terminal-Strom. // Mit unserer **Eigenentwicklung** fallen nach der Initialinvestition *null Euro Lizenzgebühren* an. Wir haben die laufenden Betriebskosten exakt kalkuliert: *[Betont die Zahlen]* Ein sparsamer lokaler Datenbank-Server im 24/7-Betrieb benötigt ca. 83 € Strom im Jahr, die 3 Touch-Terminals im Schichtbetrieb ca. 120 €. Zusammen mit einer Backup-Pauschale von 240 € belaufen sich die laufenden Gesamtkosten auf nur **ca. 443 € pro Jahr**. // Gegenüber der SaaS-Lösung spart der Betrieb jedes Jahr **1.801 € an reinen Betriebskosten** ein! *[Geste auf das Fazit unten]* In Kombination mit der vollen Datenhoheit nach DSGVO war die Entscheidung für ‚Make‘ eindeutig.“

? Prüferfrage: „Wie berechnen sich die 203 € Stromkosten pro Jahr konkret?“

💡 Antwort: „Wir rechnen mit einem energieeffizienten Mini-Server (25 W Dauerbetrieb 24/7 = 219 kWh/a) und 3 Touch-Terminals (je 35 W × 10 h/Tag × 300 Betriebstage = 315 kWh/a). Bei einem gewerblichen Strompreis von 0,38 €/kWh ergeben 534 kWh exakt 202,92 € pro Jahr.“

 Wort-für-Wort-Redeskript:

[Blick zur linken Phasenübersicht, dann zur rechten Tabelle]

„Auf dieser Folie sehen Sie unsere Ressourcenplanung und die vollständige IHK-Zuschlagskalkulation: // Das Projekt gliedert sich in sechs Phasen mit dem Schwerpunkt auf der Implementierung, die 50 % des 320-Stunden-Budgets beansprucht. // Bei einem durchschnittlichen internen Stundensatz von 45 € ergeben sich **14.400 € Personalkosten**. // Besonders wichtig war uns die exakte Erfassung der **Sach- und Betriebskosten von 500 €**: *[Zeigt auf die blaue Infobox links unten]* Hierin sind 28 € direkte Stromkosten für die 4 Entwicklungs-PCs und den Testserver enthalten – exakt 64 kWh zu 0,40 €. Hinzu kommen 172 € anteilige Hardware-Abschreibung, 200 € für Software-Tools und Repo-Hosting sowie 100 € Puffer. // Daraus resultieren **14.900 € Herstellkosten**. // Mit einem Gemeinkostenzuschlag von 20 % – also 2.980 €, worüber anteilig Raummiete, Heizung und Gebäudestrom gedeckt werden – liegen die **Selbstkosten bei 17.880 €**. // Zuzüglich 10 % kalkuliertem Gewinnzuschlag ergibt sich unser marktfähiger Nettoangebotspreis von **19.668 €** bzw. 23.404,92 € brutto. // *[Moderationsbrücke & Übergabe an Sprecher 2 / Block 2]* Damit stand der wirtschaftliche Rahmen fest. Wie wir diese Anforderungen nun prozessual und datenbankseitig modelliert haben, zeigt Ihnen jetzt der zweite Teil.“

? Prüferfrage: „Warum wurden Stromkosten für Entwicklungs-PCs als Sachkosten angesetzt und nicht in den Gemeinkosten?“

💡 Antwort: „Wir haben sauber getrennt: Der direkte Stromverbrauch der Entwickler-Workstations und des Testservers (64 kWh = ca. 28 €) ist dem Projekt direkt zurechenbar. Die allgemeinen Grundlasten wie Büro-Beleuchtung, Heizung, Gebäudestrom und Serverraum-Kühlung fließen sachgerecht in den Gemeinkostenzuschlag von 20 % ein.“

**Wort-für-Wort-Redeskript:**

[Sprecher 2 übernimmt mit Blickkontakt und Verweis auf den Quellcode in der rechten IDE-Box]

„Vielen Dank. // Schauen wir uns nun an, wie der Geschäftsprozess direkt in unserer Softwarearchitektur implementiert ist. // Auf der linken Seite sehen Sie die vier sequentiellen Prozessphasen: *[Zeigt auf die linken Kacheln]* 1.

Bestellaufnahme: Am Tisch wählt die Servicekraft den Tisch aus und erfasst Artikel – die Bestellung startet mit dem Status ‚aufgegeben‘. // 2. **Automatisches Routing:** Das System trennt Posten automatisch nach Artikelkategorie: Speisen gehen auf das Display der Küche, Getränke direkt an die Bar. // 3. **Zubereitung & Quittierung:** Die Küche setzt den Zustand auf ‚in Bearbeitung‘ und bei Fertigstellung auf ‚fertig‘. // 4. **Servieren & Abrechnung:** Nach dem Servieren erfolgt die Gesamtrechnung und automatische Freigabe des Gasttisches. // *[Blick auf die rechte IDE-Box mit Java-Code]* Rechts sehen Sie die programmierte Zustandsmaschine in `DatabaseManager.updateBestellstatus()`: Statusübergänge werden parametrisiert und synchron über ein `PreparedStatement` in die Datenbank geschrieben. Das stellt sicher, dass Service, Küche und Bar jederzeit exakt denselben, konsistenten Auftragsstatus sehen.“

? Prüferfrage: „Wie verhindert Ihre Software, dass ungültige Statusübergänge stattfinden (z. B. von 'fertig' zurück auf 'aufgegeben')?“

💡 Antwort: „Wir steuern das über ein Finite State Machine Pattern im Backend: Vor dem Aufruf von `updateBestellstatus()` validiert die Service-Logik, ob der Zielstatus ein erlaubter Folgezustand des aktuellen Status ist. Zusätzlich sichert ein SQL-CHECK-Constraint auf Spaltenebene nur die definierten Enum-Statuswerte ab.“

? Prüferfrage: „Wie wird das Routing zwischen Küche und Bar technisch realisiert?“

💡 Antwort: „Beim Auslesen der offenen Positionen filtert die GUI über das Attribut `kategorie` der Tabelle `artikel`: Das Küchen-Display filtert auf 'Vorspeisen', 'Hauptspeisen' und 'Desserts', während die Bar alle Positionen der Kategorie 'Getränke' anzeigt.“

 Wort-für-Wort-Redeskript:

[Blick auf die drei oberen Normalform-Karten, dann auf den SQL-Code rechts]

„Ein zentrales Qualitätsmerkmal unseres Projekts ist das relationale Datenmodell in der **3. Normalform**. // Wir haben Redundanzen und Anomalien systematisch eliminiert: In der 1NF wurden alle Attribute atomisiert. In der 2NF wurden Positionsdaten aus der Bestellung ausgelagert. In der 3NF wurden transitive Abhängigkeiten beseitigt – beispielsweise hängen Artikelkategorien und Mitarbeiterrollen nur vom eigenen Schlüssel ab. // *[Zeigt auf den SQL-Code in der rechten IDE-Box]* Rechts sehen Sie unsere Kernrelation `bestellposition`: Sie löst die n:m-Beziehung zwischen `bestellung` und `artikel` über zwei Fremdschlüssel sauber auf. // Achten Sie bitte auf Zeile 6: `einzelpreis DECIMAL(10,2) NOT NULL`. // Wir **historisieren den Einzelpreis** direkt in der `Bestellposition`! *[Betont diesen Punkt stark]* Warum? Würde der Gastronom nächsten Monat die Speisekartenpreise in der Tabelle `artikel` um 50 Cent anheben, blieben historische Rechnungen und Buchhaltungsabschlüsse dadurch absolut unverfälscht. // Das Ergebnis: **0 % Datenredundanz** und maximale Revisionssicherheit.“

? **Prüferfrage: „Warum ist ein historisierter Einzelpreis in der Bestellposition kein Verstoß gegen die 3NF?“**

💡 **Antwort:** „Es ist kein Verstoß, sondern fachliche Notwendigkeit: Der Preis in der Tabelle `Artikel` ist der *aktuelle* Katalogpreis, während der Einzelpreis in der `Bestellposition` ein *zeitpunktbezogenes Faktum* des Verkaufsabschlusses darstellt. Er hängt voll vom Primärschlüssel der Position zum Bestellzeitpunkt ab.“

? **Prüferfrage: „Wann würden Sie ON DELETE CASCADE und wann RESTRICT einsetzen?“**

💡 **Antwort:** „Bei abhängigen Detaildatensätzen wie `bestellposition` nutzen wir `CASCADE`, da eine Position ohne Bestellung keinen Sinn ergibt. Bei Stammdaten wie `tisch` oder `artikel` nutzen wir `RESTRICT`, um versehentliches Löschen von Tischen mit aktiven offenen Bestellungen zu verhindern.“

**Wort-für-Wort-Redeskript:**

[Zeigt auf die beiden Code-Boxen: links Transaktionslogik, rechts DAO-Zugriff]

„Auf Softwareebene setzen wir auf modernes **Java 17** und eine strikte Schichtentrennung nach dem **DAO-Entwurfsmuster (Data Access Object)**. // *[Blick auf die linke Code-Box: OrderTransaction.java]* In der linken Box sehen Sie unser Transaktionsmanagement nach den **ACID-Prinzipien**: Über `conn.setAutoCommit(false)` leiten wir eine atomare Transaktion ein. Die Erstellung des Bestellkopfs, das iterative Einfügen aller Positionen und das Setzen des Tischstatus auf ‚belegt‘ finden in einer unteilbaren Einheit statt. // Tritt während des Vorgangs ein Fehler oder ein Netzwerkabbruch auf, greift sofort `conn.rollback()` im Catch-Block. Dadurch ist ein inkonsistenter Datenbankzustand technisch ausgeschlossen. // *[Blick auf die rechte Code-Box: DatabaseManager.java]* Rechts sehen Sie unsere DAO-Abfrage: Über relationale SQL-JOINS werden Bestelldaten, Tischnummern und Servicemitarbeiter konsolidiert ausgelesen. // Die Benutzeroberfläche bleibt dadurch vollkommen frei von SQL-Syntax, und dank PreparedStatement sind **SQL-Injections** technisch zu 100 % ausgeschlossen.“

? Prüferfrage: „Was bedeuten die ACID-Prinzipien konkret für Ihren Bestellvorgang?“

Antwort: „**A (Atomarität)**: Entweder wird die gesamte Bestellung mit allen Positionen und Tischstatus gespeichert oder gar nichts. **C (Konsistenz)**: Fremdschlüssel und Constraints bleiben stets integer. **I (Isolation)**: Parallele Buchungen sehen keine unvollständigen Zwischenstände (Dirty Reads). **D (Dauerhaftigkeit)**: Nach `commit()` sind die Daten persistent auf der Festplatte gesichert.“

? Prüferfrage: „Wie schützen Prepared Statements technisch vor SQL-Injection?“

Antwort: „Bei einem PreparedStatement wird das SQL-Gerüst vorkompiliert an die Datenbank gesendet. Die über `ps.setInt()` oder `ps.setString()` übergebenen Parameter werden als reine Literalwerte behandelt und können die logische Syntax des SQL-Parsers niemals manipulieren.“

**Wort-für-Wort-Redeskript:**

[Geste auf die drei Funktionskacheln und den DIN-Standard unten]

„Die Benutzeroberfläche wurde in Java Swing realisiert und konsequent nach den Grundsätzen der **DIN EN ISO 9241-110 für Dialogsysteme** gestaltet. // Im hektischen Gastronomiealltag entscheidet jede Sekunde. Daher haben wir drei Prinzipien umgesetzt: // 1. **Aufgabenangemessenheit**: Das 2-Klick-Prinzip. Tisch anklicken, Artikel auswählen, Bon absenden – fertig. Keine verschachtelten Menüs. // 2. **Fehlertoleranz**: Das UI fängt ungültige Eingaben sofort im Event-Listener ab. Es ist unmöglich, negative Preise oder Buchstaben in Mengenangaben einzugeben. // 3. **Erwartungskonformität**: Farbliche Statusanzeigen nach Ampelprinzip – Grün für freie Tische, Rot für belegte, Gelb für fertige Bestellungen. // *[Moderationsbrücke & Übergabe an Sprecher 3 / Block 3]* Damit ist die Brücke von der Theorie zur Praxis geschlagen. Wie wir die Software automatisiert getestet und nach ISO 27001 abgesichert haben, erläutert Ihnen nun mein Kollege im dritten Block.“

? Prüferfrage: „Wie haben Sie in Java Swing sichergestellt, dass die Oberfläche bei Datenbankabfragen nicht einfriert?“

Antwort: „Längere Datenbankoperationen und Hintergrundabfragen werden vom Event Dispatch Thread (EDT) entkoppelt – beispielsweise über `SwingWorker` oder asynchrone Threads –, damit die Benutzeroberfläche jederzeit flüssig und bedienbar bleibt.“

 Wort-für-Wort-Redeskript:

[Sprecher 3 übernimmt, deutet auf den grünen Badge links und den Testcode rechts]

„Vielen Dank. // Qualität entsteht nicht durch manuelle Stichproben am letzten Tag, sondern durch systematische Verifikation. // Wir haben eine automatisierte Java-Testsuite in DatabaseTest.java entwickelt. Wie Sie links sehen, wurden alle fünf Testfälle (TC-01 bis TC-05) mit einer **Erfolgsquote von 100 %** bestanden. // *[Blick auf den Code rechts]* Hier sehen Sie zwei unserer Härtetests: In Zeile 7 schleusen wir gezielt einen böartigen SQL-Injection-String ein – ' DROP TABLE bestellung; --. Der Test validiert, dass der String maskiert wird und keine Tabellen gelöscht werden. // In Zeile 13 prüfen wir die referentielle Integrität: Ein Versuch, eine Bestellung auf einen nicht existierenden Tisch 999 zu buchen, wird von der Datenbank sofort mit einem Constraint-Fehler abgewiesen. // Diese Testsuite läuft vor jeder Auslieferung und garantiert 100 % Regressionsschutz.“

 Prüferfrage: „Wie trennen Sie Testdaten von echten Produktivdaten?“

 **Antwort:** „Die automatisierte Testsuite initialisiert für jeden Testlauf eine isolierte Testdatenbank (bzw. eine In-Memory-Instanz), führt das Schema aus 13-datenbank-schema.sql aus und rollt alle Test-Transaktionen am Ende wieder zurück.“

 Wort-für-Wort-Redeskript:

[Zeigt auf die drei ISO-Kontrollkarten links und die Kryptographie-Funktion rechts]

„Als IT-Fachkräfte tragen wir Verantwortung für die Betriebssicherheit unseres Kunden. Daher haben wir uns am internationalen Standard **ISO/IEC 27001** orientiert: // Erstens **Zugriffssteuerung (RBAC nach Annex A.5 & A.8)**: Servicekräfte können ausschließlich Tische buchen. Preise und Personalstammdaten können nur mit Administratorrechten eingesehen und modifiziert werden. // Zweitens **Kryptographie (Annex A.8.24)**: *[Zeigt auf den Code rechts]* Wir speichern Passwörter niemals im Klartext, sondern nutzen den anerkannten Standard PBKDF2WithHmacSHA256 mit 65.536 Iterationen und kryptographischem Zufalls-Salt. Das macht Rainbow-Table-Angriffe unmöglich. // Drittens **Datensicherung (Annex A.8.12)**: Durch automatisierte SQL-Dumps garantieren wir im Havariefall eine Wiederanlaufzeit – ein **RTO** – von unter 60 Minuten. Zudem gewährleisten wir durch Datensparsamkeit volle DSGVO-Konformität.“

 Prüferfrage: „Was ist der Unterschied zwischen Verschlüsselung und Hashing, und warum nutzt man Salt?“

 **Antwort:** „Verschlüsselung ist bijektiv und umkehrbar (mit Schlüssel). Hashing ist eine Einwegfunktion. Das Salt (Zufallswert) verhindert Rainbow-Table-Angriffe, da identische Passwörter zweier Mitarbeiter völlig unterschiedliche Hashwerte erzeugen.“

 Wort-für-Wort-Redeskript:

[Blick auf die drei großen KPI-Kacheln oben]

„Kommen wir zum Projektcontrolling und dem Soll-Ist-Vergleich: // Auf Makroebene haben wir eine Punktlandung erreicht: **100 % Funktionsumfang** – alle 26 Muss- und Sollanforderungen wurden umgesetzt. Das Zeitbudget von 320 Stunden und die kalkulierten Herstellkosten von 14.900 € wurden exakt eingehalten. // *[Zeigt auf die Abweichungsanalyse unten in der Tabelle]* Im Detail gab es jedoch eine Planabweichung, die wir professionell gesteuert haben: Die Implementierung des interaktiven Swing-Tabellenrenderers erforderte 8 Stunden mehr als im Entwurf geschätzt. Da wir jedoch in Phase 3 durch den Einsatz präziser CASE-Tools beim relationalen Datenbankdesign 8 Stunden schneller waren als geplant, konnten wir diesen Verzug intern **vollständig kompensieren** – ohne Budget- oder Termineffekte für den Kunden. // Zusätzlich konnten wir im Budgetrahmen eine automatisierte Testsuite und die ISO 27001-Dokumentation fertigstellen.“

? Prüferfrage: „Wie hätten Sie reagiert, wenn die 8 Stunden Verzögerung nicht kompensierbar gewesen wären?“

💡 Antwort: „Wir hätten nach MoSCoW-Priorisierung entschieden: Optionale Kann-Kriterien (z. B. zusätzliche Filterfunktionen im Admin-Dashboard) wären in eine Folgephase verschoben worden, um den Liefertermin der Kernfunktionen für Service und Küche niemals zu gefährden.“

 Wort-für-Wort-Redeskript:

[Blick zu den Prüfern, offene Gestik auf die beiden Spalten]

„Was nehmen wir aus diesem Projekt mit? // Unsere wichtigste Erkenntnis: Die strikte Schichtentrennung über das DAO-Pattern hat es uns ermöglicht, völlig parallel an der Datenbank und der Benutzeroberfläche zu arbeiten, ohne uns zu blockieren. Zudem hat das parallele Führen unseres Projekttagebuchs jeglichen Dokumentationsstress vor der Abgabe vermieden. // *[Blick auf die Roadmap rechts]* Unser System ist modular und für die Zukunft gerüstet: In **Phase 2** binden wir mobile Kellner-Tablets über ein responsives Webinterface an. In **Phase 3** erfolgt die Anbindung einer zertifizierten Technischen Sicherheitseinrichtung (TSE) nach der **Kassensicherungsverordnung (KassenSichV)** für den gesetzestkonformen Belegdruck. Und in **Phase 4** statten wir die Küche mit einem vollwertigen Kitchen-Display-Touchscreen und akustischer Fertigmeldung aus.“

 Prüferfrage: „Was fordert die KassenSichV (Kassensicherungsverordnung) für Ihr System?“

 **Antwort:** „Die KassenSichV verlangt eine zertifizierte Technische Sicherheitseinrichtung (TSE – Hardware-USB oder Cloud-TSE), die jede Transaktion kryptographisch signiert, einen manipulationssicheren Prüfwert generiert und die Belegausgabepflicht erfüllt.“

 Wort-für-Wort-Redeskript:

[Alle drei Teammitglieder treten nebeneinander vor den Tisch, aufrechte Haltung, Blickkontakt]

„Fassen wir zusammen: // Mit ‚Smart Restaurant‘ haben wir ein digitales, wirtschaftliches und sicheres Gesamtsystem geschaffen, das den papierbasierten Gastronomiebetrieb erfolgreich in das digitale Zeitalter überführt. // Alle vertiefenden Dokumente – von der Entwicklerdokumentation über das Benutzerhandbuch bis hin zur ISMS-Sicherheitsdokumentation und den SQL-Dumps – liegen Ihnen vollständig vor. // *[Verbeugung / Kopfnicken zum Ausschuss]* Wir bedanken uns ganz herzlich für Ihre Aufmerksamkeit und freuen uns nun auf Ihre Fragen im IHK-Fachgespräch.“

 Typische Eröffnungsfrage im Fachgespräch: „Was war für Sie persönlich die größte Herausforderung im Projekt?“ **Musterantwort (je nach Rolle):**

- **Wirtschaftlichkeit/Leitung:** „Die realistische Kostenstellen-Umlage von Strom- und Gemeinkosten und das Aufsetzen der agilen Sprints.“
- **Architektur/DB:** „Das synchrone Zustandsmanagement zwischen Service und Küche sowie die 3NF-Modellierung ohne Datenverlust.“
- **Qualität/Security:** „Die lückenlose Absicherung gegen SQL-Injections und das Erstellen automatisierter Testfälle mit Rollbacks.“

3. Souveräne Antwort-Taktik für das IHK-Fachgespräch

Herausforderung	Erprobte Reaktions- & Antwortstrategie
Eine Prüferfrage nicht verstanden:	Ruhig und professionell nachfragen: „Bezieht sich Ihre Frage auf die Kapselung in der Java-Datenbankschicht oder auf die Konsistenzprüfung im relationalen SQL-Schema?“ Das beweist Fachkompetenz und verschafft 10 Sekunden Denkzeit.
Ein Detail nicht parat:	Niemals raten oder erfinden! Brücken zum Architekturprinzip bauen: „Die exakte Syntax dieser JDBC-Methode habe ich gerade nicht auswendig vor Augen, aber das architektonische Prinzip dahinter ist die Entkopplung über...“
Ein Teamkollege stockt:	Als Team glänzen: Nicht unhöflich ins Wort fallen, sondern nach 3 Sekunden sanft ergänzen: „Ergänzend zu dem, was Kollege X gerade erläutert hat, haben wir im Datenbank-Layer zusätzlich darauf geachtet, dass...“
Kritische Rückfrage zu einer Entscheidung:	Sachlich begründen und Offenheit zeigen: „Das ist ein absolut berechtigter Einwand. Für das 10-Tage-Projekt war unsere Lösung der pragmatischste Weg; in einer Enterprise-Umgebung würden wir genau Ihren Ansatz mit einem Connection-Pool wählen.“